# Stochastic Windows and Multi-Bucket Assignment

Macharia Barii

November 3, 2026

**Abstract**

Two extensions reveal something important: stochastic windows are another form of addressability, and multi-route assignment breaks the bucket decomposition theorem in a way that is repairable but changes the algorithm. This paper treats the customer's window as a probability distribution rather than a guaranteed interval, shrinks the schedule window by the recipient's uncertainty, and adds a variance-buffer term to the allocation rule. It then lets a drop be assigned to any of several feasible buckets, derives the rigidity externality and the marginal rigidity cost, and gives an assignment algorithm that places each rigid drop where it does the least additional harm. The bucket decomposition theorem survives only if the externality is paid for. The result is a single unified allocation formula that contains all previous forms as special cases, and a customer-facing menu rather than a price.

# Part I
# Stochastic Windows

## 1 The window is a distribution, not an interval

When a customer says "between 9 and 11," they usually mean something weaker. They might be there at 9, might not be there until 10:30. The window is a probability distribution over availability, not a guaranteed interval.

**Definition 1.** *Drop $j$ has a window distribution with mean availability interval $[e_j, \ell_j]$ and standard deviation $\sigma_{w_j}$. The probability of successful delivery at arrival time $t$ is $q_j(t)$, peaked at the centre of $[e_j, \ell_j]$ and declining toward the edges.*

For pricing purposes, the key quantity is $\sigma_{w_j}$—the uncertainty around when the recipient is actually available.

## 2 The effective schedule window

The deterministic schedule window $S_j = L_j - E_j$ is computed by the forward/backward pass. Under stochastic windows, it shrinks because the route must reserve buffer for the recipient's uncertainty.

At service level $1 - \alpha$:

$$S_j^{\text{eff}} = S_j - z_{1-\alpha} \cdot \sigma_{w_j} \tag{1}$$

where $z_{1-\alpha}$ is the normal quantile ($z = 1.645$ for 95%, $z = 1.28$ for 90%, $z = 0.84$ for 80%).

If $S_j^{\text{eff}} \leq 0$, the drop is infeasible at that service level. The correct response is to lower the service level for that drop, widen the window, or reject—not to price it higher, because no price makes an infeasible route feasible.

The service level is a strategic choice. Higher confidence means fewer feasible routes and higher prices; lower confidence means more volume but more failures. The engine must offer

the customer a menu: "at 95% confidence, your window makes the route infeasible; at 80% confidence we can serve you for X."

## 3 The four-component allocation

The shared cost is now allocated across four components:

$$s_j = \alpha \cdot \frac{\mu_j}{\sum_k \mu_k} + \beta \cdot \frac{\Delta d_j}{D_R} + \gamma \cdot \frac{1/S_j^{\text{eff}}}{\sum_k 1/S_k^{\text{eff}}} + \delta \cdot \frac{\sigma_{w_j}^2}{\sum_k \sigma_{w_k}^2} \tag{2}$$

with $\alpha + \beta + \gamma + \delta = 1$. Default: $\alpha = 0.30$, $\beta = 0.20$, $\gamma = 0.30$, $\delta = 0.20$.

The new fourth term is the variance buffer share. Note it uses $\sigma^2$, not $\sigma$—because the route's total buffer grows with the square root of the sum of variances, so the marginal buffer cost of drop $j$ is proportional to $\sigma_{w_j}^2$, not $\sigma_{w_j}$.

The interpretation: a drop whose window is uncertain (high $\sigma_{w_j}$) forces the route to reserve more buffer, which reduces the effective window for every drop. That drop should pay for the buffer it imposes.

## 4 Worked example

Same CBD → Thika route, but now with window uncertainty:

| Drop | Window | $\sigma_{w_j}$ | $\mu_j$ | $\Delta d_j$ |
|---|---|---|---|---|
| A | 8:00–10:00 | 10 min | 5 min | 12 km |
| B | 8:30–9:15 | 20 min | 20 min | 10 km |
| C | 9:30–12:00 | 5 min | 8 min | 14 km |

Deterministic schedule windows (from before): $S_A = 45$, $S_B = 30$, $S_C = 150$.

At 95% confidence ($z = 1.645$):

| Drop | $S_j$ | $z \cdot \sigma_{w_j}$ | $S_j^{\text{eff}}$ | Feasible? |
|---|---|---|---|---|
| A | 45 | 16.5 | 28.5 | yes |
| B | 30 | 32.9 | −2.9 | Infeasible |
| C | 150 | 8.2 | 141.8 | yes |

B is infeasible at 95% confidence. Its window is tight and uncertain. The customer must choose: accept 80% confidence, or widen the window.

At 80% confidence ($z = 0.84$):

| Drop | $S_j$ | $z \cdot \sigma_{w_j}$ | $S_j^{\text{eff}}$ | $1/S^{\text{eff}}$ | Rigidity share |
|---|---|---|---|---|---|
| A | 45 | 8.4 | 36.6 | 0.0273 | 0.248 |
| B | 30 | 16.8 | 13.2 | 0.0758 | 0.689 |
| C | 150 | 4.2 | 145.8 | 0.0069 | 0.063 |

Variance buffer shares ($\sigma^2$):

Combined shares at $\alpha = 0.30$, $\beta = 0.20$, $\gamma = 0.30$, $\delta = 0.20$:

B pays 60% of the shared cost. It is the tightest, most uncertain drop on the route, and it should be. The customer at B either accepts a higher price or gives a wider window.

Prices (shared cost = 2,400, standalone A/B/C = 200/350/250, $\hat{p} \cdot A \approx 0.99$):

| Drop | $\sigma^2_{w_j}$ | Share |
|---|---|---|
| A | 100 | 0.190 |
| B | 400 | 0.762 |
| C | 25 | 0.048 |

| Drop | Service | Distance | Rigidity | Variance | $s_j$ |
|---|---|---|---|---|---|
| A | 0.046 | 0.067 | 0.074 | 0.038 | 0.225 |
| B | 0.182 | 0.056 | 0.207 | 0.152 | 0.597 |
| C | 0.073 | 0.078 | 0.019 | 0.010 | 0.180 |

Compare to the deterministic-window case: A falls 15%, B rises 16%, C falls 14%. B pays for its uncertainty; C benefits from its predictability.

## 5 The unification: window variance is addressability

This is the deep connection. A recipient who is reliably available at a stated time is well-addressed. A recipient who is available only vaguely is poorly-addressed. The $\sigma_{w_j}$ term is a measure of dynamic addressability—the same $a^D$ component that accumulates through the ledger.

The practical consequence: every successful delivery at a given time-of-day updates the estimate of $\sigma_{w_j}$ for that recipient. A recipient who has accepted 20 deliveries in the 9–11am window with a 3-minute standard deviation has $\sigma_{w_j} = 3$—well-addressed. A recipient who has accepted 3 deliveries with a 45-minute standard deviation has $\sigma_{w_j} = 45$—poorly-addressed.

This means window variance is learnable, and the ledger is the mechanism. The recipient trust graph from the B2C expansion carries $\sigma_{w_j}$ alongside its first-attempt history. Over time, $\sigma_{w_j}$ falls, the effective schedule window widens, the rigidity share falls, and the price falls. This is the flywheel applied to time windows specifically.

# Part II
# Multi-Route Assignment

## 6 The problem: buckets are not independent

The bucket decomposition theorem said that each bucket can be solved independently. That was true because we assumed each drop belongs to exactly one bucket—its corridor and time constraint determine which bucket it is in. That assumption breaks when a drop can go to multiple buckets.

If drop B can travel on the 7:30 Thika run or the 9:30 Thika run, then assigning it to one bucket affects the other. The buckets are coupled through the drops that could go to either.

The bucket decomposition theorem does not fail—it fails to be applicable. The problem is no longer "decompose into independent buckets"; it is "assign each drop to a bucket such that no bucket's rigidity exceeds a threshold, and the assignment is stable."

## 7 The rigidity externality

The key concept is the externality: when a rigid drop joins a bucket, it reduces the schedule windows of every other drop in that bucket.

| Drop | Standalone | Allocated shared | Net $n_j$ | Client price |
|---|---|---|---|---|
| A | 200 | 540 | 747 | 866 |
| B | 350 | 1,433 | 1,801 | 2,089 |
| C | 250 | 432 | 689 | 799 |

Formally, for drop $j$ assigned to bucket $b$ with existing set $S_b$:

$$\Delta_{\text{rigid}}(j, b) = \sum_{k \in S_b} \left[ S_k^{\text{before}} - S_k^{\text{after}} \right] \cdot v_{\text{flex}} \tag{3}$$

where $v_{\text{flex}}$ is the value of one minute of schedule flexibility on that bucket (approximately the cost of the failure probability it induces).

This is a negative externality. Drop $j$ does not pay it directly; every other drop in the bucket pays it. The correct response is to assign $j$ to whichever bucket has the lowest marginal externality.

## 8 The marginal rigidity cost

**Definition 2.** *The marginal rigidity cost of assigning drop $j$ to bucket $b$ is:*

$$\Delta_b(j) = \sum_{k \in S_b} (\mu_j + \tau_{j,k}) \cdot v_{flex} + \sigma_{w_j}^2 \cdot v_{buf} \tag{4}$$

*where the first term is the schedule window reduction imposed on every other drop (approximately $\mu_j$ plus travel time), and the second term is the buffer cost from variance.*

The assignment rule: assign $j$ to the bucket where $\Delta_b(j)$ is lowest, subject to feasibility at the bucket's service level.

## 9 The assignment algorithm

```
def assign_to_bucket(drop, candidate_buckets):
    """
    Given a drop and the buckets it could feasibly join,
    assign to the bucket where the marginal rigidity cost is lowest.
    """
    best = None
    for b in candidate_buckets:
        if not feasible(drop, b):
            continue
        # compute schedule windows with and without drop
        S_before = compute_schedule_windows(b.drops, b.plan)
        S_after = compute_schedule_windows(b.drops + [drop], b.plan)

        # marginal schedule reduction for each existing drop
        rigidity_cost = sum(
            (S_before[k] - S_after[k]) * value_of_flexibility(b)
            for k in b.drops
        )
        # marginal variance buffer
        buffer_cost = (variance(drop.window) * value_of_buffer(b))

        total = rigidity_cost + buffer_cost

        if best is None or total < best.cost:
```

```
            best = BucketChoice(b, total)

    if best is None:
        return Verdict.PREPARE
    if best.cost > drop.margin * MAX_EXTERNALITY_SHARE:
        return Verdict.PASS
    best.bucket.add(drop)
    return Verdict.PROVE
```

The `MAX_EXTERNALITY_SHARE` is the fraction of the drop's margin that can be consumed by its externality— typically 0.3–0.5. Above this, the drop is not paying its way; the bucket's other drops are subsidising it, and the correct response is PASS.

## 10 Worked example: B chooses between two routes

**Setup.** Drop B (window 8:30–9:15, $\sigma_w = 20$, $\mu = 20$, $\Delta d = 10$) can travel on:

- Route 1 (departure 7:30): drops A (8:00–10:00, $S_A = 45$), C (9:30–12:00, $S_C = 150$)
- Route 2 (departure 9:30): drops D (10:00–11:00, $S_D = 60$), E (10:30–12:30, $S_E = 120$)

**Route 1: B fits between A and C.** Without B: $S_A = 60$, $S_C = 175$. With B: $S_A = 45$, $S_C = 150$. Marginal reduction on A: $60 - 45 = 15$ min. Marginal reduction on C: $175 - 150 = 25$ min. Total schedule externality on route 1: 40 minutes.

**Route 2: B fits before D and E.** Without B: $S_D = 90$, $S_E = 150$. With B: $S_D = 60$, $S_E = 120$. Marginal reduction on D: $90 - 60 = 30$ min. Marginal reduction on E: $150 - 120 = 30$ min. Total schedule externality on route 2: 60 minutes.

Route 1 has lower externality (40 min vs. 60 min). B should be assigned to route 1.

Why? Route 1's drops already have constraints; B's addition is less disruptive because A and C are already partially constrained. Route 2's drops are more flexible; B's addition would consume more of that flexibility. The rigid drop belongs where there are already other rigid drops, because the marginal cost of rigidity is lower where rigidity is already present.

This is counterintuitive and important. The natural instinct is "put the rigid drop on the flexible route." The correct answer is the opposite—put it where it does the least additional harm.

## 11 Equilibrium and convergence

The assignment rule is a potential game—each drop's cost depends on the bucket it joins, and the total potential is the sum of all drops' costs. Potential games have a Nash equilibrium, and a simple best-response dynamic converges to it.

The equilibrium is efficient in a specific sense: no drop can reduce its own cost by switching, which means the externality is internalised at the margin. This is the same logic as Wardrop equilibrium in traffic—drops distribute across buckets so that no one gains from switching, which is not the social optimum but is close to it and computationally reachable.

The convergence condition: the equilibrium exists and is unique if the marginal rigidity cost is increasing in bucket load. This is satisfied whenever $\mu_j + \tau_{j,k} > 0$, which is always true.

## 12 Repairing the bucket decomposition theorem

The bucket decomposition theorem can be restored with a modification: replace the fixed bucket cost with the marginal externality cost.

**Theorem 1** (Modified bucket decomposition)**.** *If each drop's assigned bucket is chosen to minimise $\Delta_b(j)$ subject to feasibility, and if the marginal externality $\Delta_b(j)$ is monotone in bucket load, then:*

1. *The assignment problem has a unique equilibrium.*
2. *The equilibrium is computable in $O(n \cdot B \cdot \log n)$ where $B$ is the number of buckets.*
3. *At equilibrium, the total externality across all buckets is no more than twice the minimum achievable.*

In plain terms: the decomposition still works, but you have to pay for the externality. The bucket remains a tractable object; the choice of which bucket is the new decision variable.

# Part III
# The Unified Formula

## 13 The master allocation equation

Combining the four components with the externality:

$$s_j = \alpha \cdot \frac{\mu_j}{\sum_k \mu_k} + \beta \cdot \frac{\Delta d_j}{D_R} + \gamma \cdot \frac{1/S_j^{\text{eff}}}{\sum_k 1/S_k^{\text{eff}}} + \delta \cdot \frac{\sigma_{w_j}^2}{\sum_k \sigma_{w_k}^2} + \epsilon \cdot \frac{\Delta_b(j)}{\sum_k \Delta_b(k)} \tag{5}$$

where $\epsilon$ is the externality share and $\Delta_b(j)$ is the marginal rigidity cost of the drop in its chosen bucket.

When there are no stochastic windows, no multi-route choice, and no time windows at all, all three new terms ($\gamma$, $\delta$, $\epsilon$) go to zero, and the formula reduces to the original two-component rule. The general formula contains all previous forms as special cases.

## 14 What the engine needs

The bucket algorithm must be extended with four new steps:

1. **Window variance estimation.** The ledger tracks $\sigma_{w_j}$ per recipient per time-of-day. Initial estimate from the customer's stated window; updated from first-attempt history.
2. **Service-level menu.** Each bucket offers a set of service levels (80%, 90%, 95%), each with a price. The customer chooses.
3. **Multi-bucket candidate computation.** For each drop, identify the buckets it could feasibly join, and compute marginal rigidity for each.
4. **Externality-capped assignment.** Assign to the lowest-externality bucket, subject to the constraint that the drop's margin exceeds its externality share.

## 15 What the diagnostic reveals

The utilisation diagnostic splits into three cases under stochastic windows:

The variance column separates the two window-constraint cases. A corridor where deliveries are consistently late because customers are unreliable is a different problem from a corridor where deliveries are late because customers give tight windows. The first is solved by building the recipient trust graph; the second is solved by offering discounts for wider windows. The diagnostic tells you which.

| $u = R_{\text{obs}}/R(a)$ | Wait time | Variance | Diagnosis | Lever |
|---|---|---|---|---|
| Low | High | Low | Window-constrained (deterministic) | Widen windows |
| Low | High | High | Window-constrained (stochastic) | Track recipient availability; build trust |
| Low | Low | High | Consolidation-constrained | Acquire demand |
| High | Low | Low | Addressability-constrained | Verify addresses |

## 16 The operationally important consequence

The customer-facing consequence: every drop now has a menu, not a price.

"Your drop can go on the 7:30 Thika run at 95% confidence for KES X, or on the 9:30 run at 90% confidence for KES Y, or on either at 80% confidence for KES Z. If you can widen your window to 8:00–10:00, the price falls further. If you can provide a more predictable availability (we'll track it over time), the price falls further still."

This is not a discount. It is the price of the specific combination of service level, timing, and predictability that the customer is requesting. The menu makes explicit what was previously bundled and opaque.

The strategic consequence: the flywheel now has a second reinforcing loop. Better addressability (including window predictability) $\rightarrow$ lower $\sigma_w$ $\rightarrow$ wider effective schedule window $\rightarrow$ lower rigidity share $\rightarrow$ lower price or higher margin $\rightarrow$ more volume $\rightarrow$ more ledger writes $\rightarrow$ better addressability. The same flywheel, with two inputs instead of one.

## 17 The extension in one line

Under stochastic windows, the schedule window shrinks by the customer's uncertainty, the variance buffer is allocated by variance share, and the customer chooses their service level from a menu—while under multi-route assignment, the rigid drop belongs where the marginal rigidity cost is lowest, which is usually where rigidity already exists, and the bucket decomposition theorem survives only if the externality is paid for.